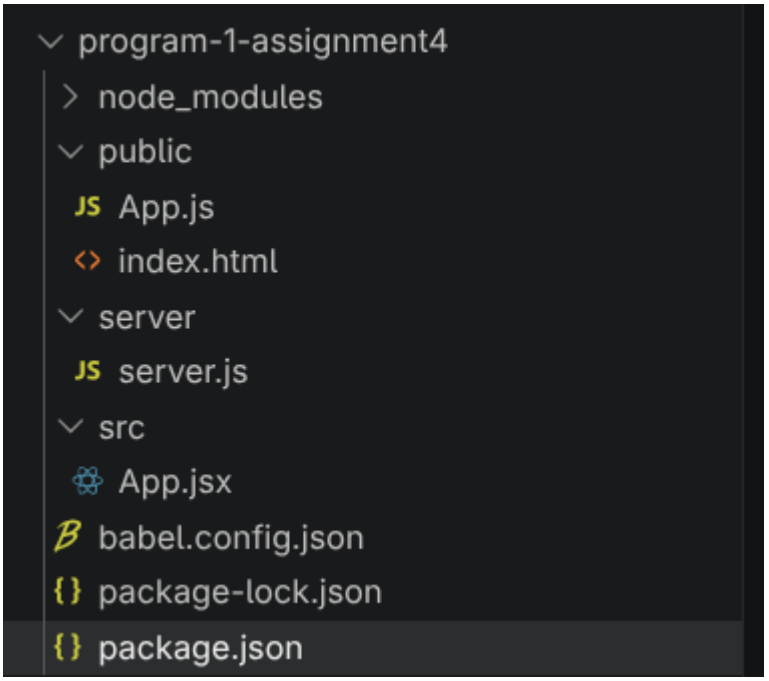
Assignment - 4 MERN Lab

15-09-2026

1. Write a React program using a class component HelloWorld that displays a greeting message for the following continents: Africa, America, Asia, Australia, and Europe. Use map() to generate the messages and join() to combine them.



package.json: Replace script code with this

```
"scripts": {
  "start": "nodemon -w server server/server.js",
  "compile": "babel src --out-dir public",
  "watch": "babel src --out-dir public --watch --verbose"
},
```

babel.config.json

```
{
  "presets": [
    [
      "@babel/preset-react",
      {
        "runtime": "classic"
      }
    ]
  ]
}
```

public/index.html

```
<!DOCTYPE html>
<html>

<head>
  <title>Hello World</title>
</head>

<body>

  <div id="root"></div>

  <script src="https://unpkg.com/react@17/umd/react.development.js"></script>
  <script
src="https://unpkg.com/react-dom@17/umd/react-dom.development.js"></script>

  <script src="App.js"></script>

</body>

</html>
```

server/server.js

```
const express = require('express');

const app = express();

app.use(express.static('public'));

app.listen(3000, function () {
  console.log('Server started at http://localhost:3000');
});
```

src/App.jsx

```
class HelloWorld extends React.Component {
  render() {
    const countries = [
      { id: 1, name: "India" },
      { id: 2, name: "Ireland" },
```

```
      { id: 3, name: "Canada" }
    ];

    const message = countries.map(c => (
      <CountryList key={c.id} country={c} />
    ));

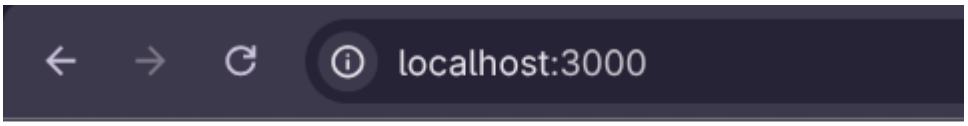
    return message;
  }
}

class CountryList extends React.Component {
  render() {
    const row = this.props.country;

    return (
      <p>Hello World from {row.name}</p>
    );
  }
}

ReactDOM.render(
  <HelloWorld />,
  document.getElementById('root')
);
```

Output:

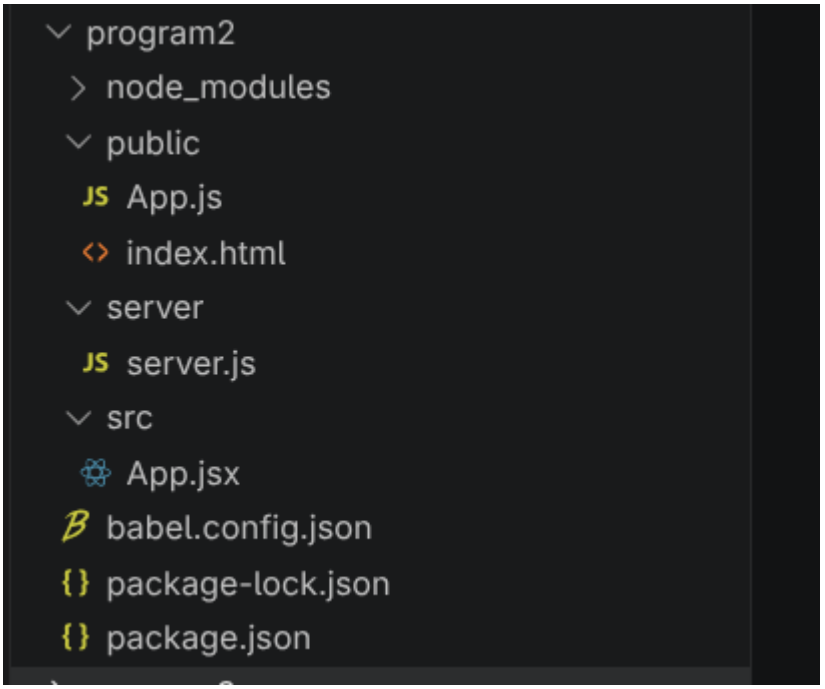


Hello World from India

Hello World from Ireland

Hello World from Canada

2. Write a React application using class components to demonstrate component composition for an Issue Tracker. Create separate IssueFilter, IssueTable, and IssueAdd components, and compose them inside an IssueList component. The IssueList component should display the title "Issue Tracker", followed by the issue filter, issue table, and issue addition form. Use React.Fragment to group the components and render the IssueList component using ReactDOM.



package.json: Replace script code with this

```
"scripts": {
  "start": "nodemon -w server server/server.js",
  "compile": "babel src --out-dir public",
  "watch": "babel src --out-dir public --watch --verbose"
},
```

babel.config.json

```
{
  "presets": [
    [
      "@babel/preset-react",
      {
        "runtime": "classic"
      }
    ]
```

```
  ]
}
```

public/index.html

```
<!DOCTYPE html>
<html>

<head>
  <title>Hello World</title>
</head>

<body>

  <div id="root"></div>

  <script src="https://unpkg.com/react@17/umd/react.development.js"></script>
  <script
src="https://unpkg.com/react-dom@17/umd/react-dom.development.js"></script>

  <script src="App.js"></script>

</body>

</html>
```

server/server.js

```
const express = require('express');

const app = express();

app.use(express.static('public'));

app.listen(3000, function () {
  console.log('Server started at http://localhost:3000');
});
```

src/App.jsx

```
class IssueFilter extends React.Component {
  render() {
    return (<div> <input
      type="text"
      placeholder="Filter issues"
```

```
      /> <button>Filter</button> </div>
      );
    }
}

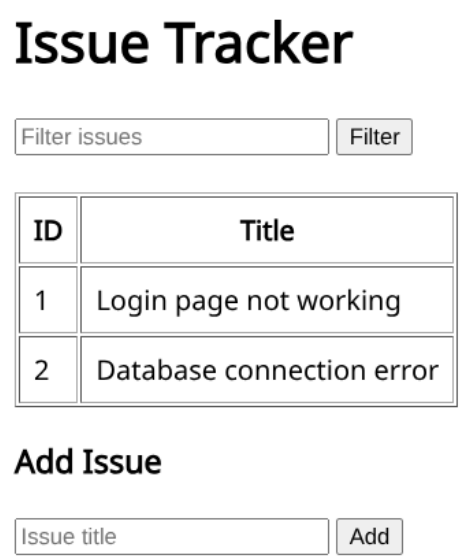
class IssueTable extends React.Component {
  render() {
    return (<table border="1" cellPadding="8"> <thead> <tr> <th>ID</th>
<th>Title</th> </tr> </thead> <tbody> <tr> <td>1</td> <td>Login page not
working</td> </tr> <tr> <td>2</td> <td>Database connection error</td> </tr>
</tbody> </table>
    );
  }
}

class IssueAdd extends React.Component {
  render() {
    return (<div> <h3>Add Issue</h3> <input
      type="text"
      placeholder="Issue title"
    /> <button>Add</button> </div>
    );
  }
}

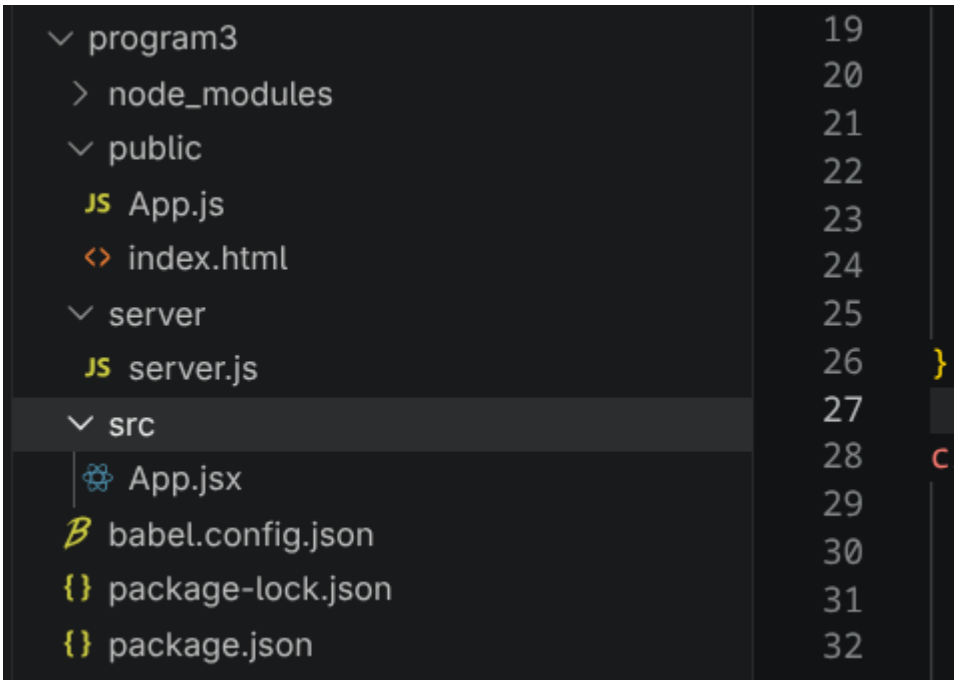
class IssueList extends React.Component {
  render() {
    return (
      <React.Fragment> <h1>Issue Tracker</h1> <IssueFilter /> <br /> <IssueTable />
<IssueAdd />
      </React.Fragment>
    );
  }
}

ReactDOM.render(<IssueList />,
  document.getElementById("root")
);
```

Output:



3. Develop a React application to demonstrate passing data from a parent component to a child component using properties (props). Create an IssueTable class component that displays an issue table with ID and Title columns. Pass the issue ID, issue title, and row styling information to an IssueRow component using props, and display two issue records in the table.



package.json: Replace script code with this

```
"scripts": {
  "start": "nodemon -w server server/server.js",
```

```
  "compile": "babel src --out-dir public",
  "watch": "babel src --out-dir public --watch --verbose"
},
```

babel.config.json

```
{
  "presets": [
    [
      "@babel/preset-react",
      {
        "runtime": "classic"
      }
    ]
  ]
}
```

public/index.html

```
<!DOCTYPE html>
<html>

<head>
    <title>Hello World</title>
</head>

<body>

    <div id="root"></div>

    <script src="https://unpkg.com/react@17/umd/react.development.js"></script>
    <script
src="https://unpkg.com/react-dom@17/umd/react-dom.development.js"></script>

    <script src="App.js"></script>

</body>

</html>
```

server/server.js

```
const express = require('express');

const app = express();

app.use(express.static('public'));
```

```
app.listen(3000, function () {
  console.log('Server started at http://localhost:3000');
});
```

src/App.jsx

```
class IssueRow extends React.Component {
  render() {
    return (
      <tr style={this.props.rowStyle}>
        <td>{this.props.id}</td>
        <td>{this.props.title}</td>
      </tr>
    );
  }
}

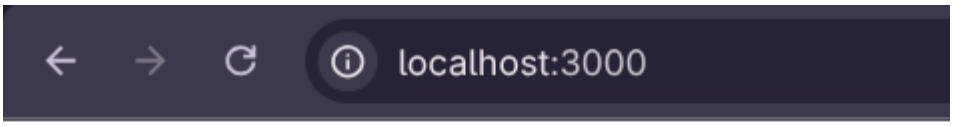
class IssueTable extends React.Component {
  render() {
    const rowStyle = {
      backgroundColor: "#f2f2f2"
    };
    return (
      <div>
        <h2>Issue Tracker</h2>
        <table border="1" cellPadding="10">
          <thead>
            <tr>
              <th>ID</th>
              <th>Title</th>
            </tr>
          </thead>
          <tbody>
            <IssueRow
              id="1"
              title="Login page not working"

              rowStyle={rowStyle}
            />
            <IssueRow
              id="2"
              title="Database connection error"
```

```
                rowStyle={rowStyle}
              />
            </tbody>
          </table>
        </div>
      );
    }
}

ReactDOM.render(
  <IssueTable />,
  document.getElementById("root")
);
```

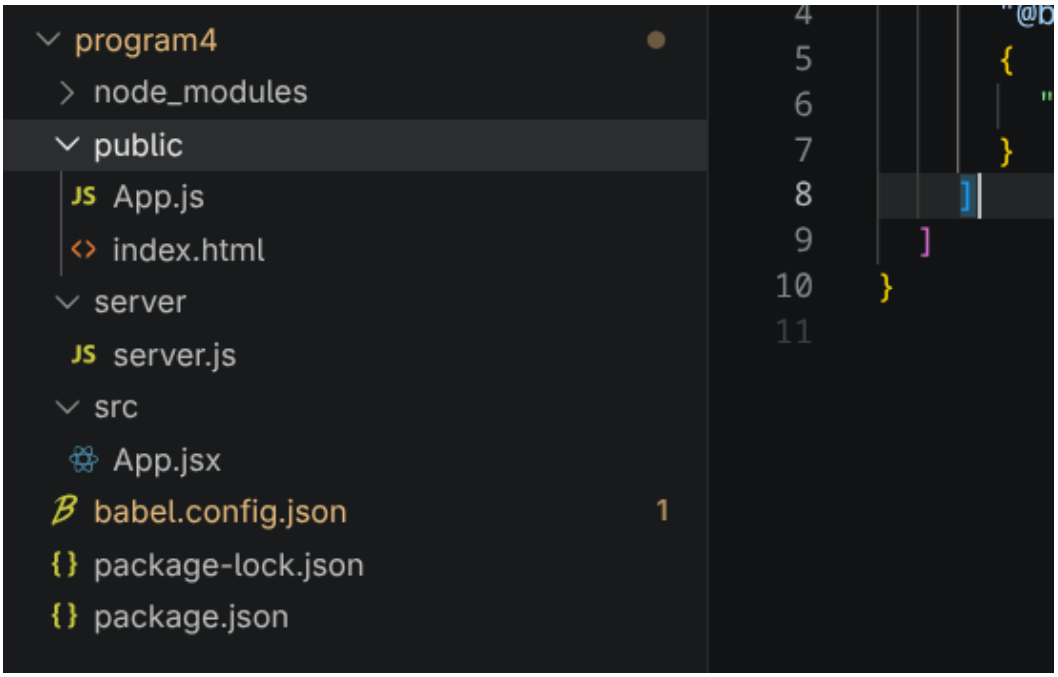
Output:



Issue Tracker

ID	Title
1	Login page not working
2	Database connection error

4. Write a React program to develop an Issue Tracker using component composition. Create IssueRow, IssueTable, IssueFilter, IssueAdd, and IssueList class components. Store issue details in an array, pass each issue object to IssueRow using props, and use map() to dynamically display all issues in a table. Render the complete Issue Tracker using ReactDOM.



package.json: Replace script code with this

```
"scripts": {
  "start": "nodemon -w server server/server.js",
  "compile": "babel src --out-dir public",
  "watch": "babel src --out-dir public --watch --verbose"
},
```

babel.config.json

```
{
  "presets": [
    [
      "@babel/preset-react",
      {
        "runtime": "classic"
      }
    ]
  ]
}
```

public/index.html

```
<!DOCTYPE html>
<html>

<head>
  <title>Hello World</title>
</head>

<body>

  <div id="root"></div>

  <script src="https://unpkg.com/react@17/umd/react.development.js"></script>
  <script
src="https://unpkg.com/react-dom@17/umd/react-dom.development.js"></script>

  <script src="App.js"></script>

</body>

</html>
```

server/server.js

```
const express = require('express');

const app = express();

app.use(express.static('public'));

app.listen(3000, function () {
  console.log('Server started at http://localhost:3000');
});
```

src/App.jsx

```
class IssueRow extends React.Component {
  render() {
    const issue = this.props.issue;

    return (
      <tr>
        <td>{issue.id}</td>
        <td>{issue.title}</td>
```

```
        </tr>
      );
    }
}


class IssueTable extends React.Component {
  render() {
    const issues = this.props.issues;

    return (
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Title</th>
          </tr>
        </thead>

        <tbody>
          {issues.map(issue => (
            <IssueRow
              key={issue.id}
              issue={issue}
            />
          ))}
        </tbody>
      </table>
    );
  }
}


class IssueFilter extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      searchText: ""
    };
  }

  handleChange = (event) => {
    this.setState({
```

```
      searchText: event.target.value
    });
  };

  handleFilter = () => {
    this.props.onFilter(this.state.searchText);
  };

  render() {
    return (
      <div>
        <input
          type="text"
          placeholder="Filter issues"
          value={this.state.searchText}
          onChange={this.handleChange}
        />

        <button onClick={this.handleFilter}>
          Filter
        </button>

        <button onClick={() => this.props.onFilter("")}>
          Clear
        </button>
      </div>
    );
  }
}


class IssueAdd extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      title: ""
    };
  }

  handleChange = (event) => {
    this.setState({
      title: event.target.value
    });
```

```
  };

  handleAdd = () => {
    if (this.state.title.trim() === "") {
      return;
    }

    this.props.onAdd(this.state.title);

    this.setState({
      title: ""
    });
  };

  render() {
    return (
      <div>
        <h3>Add Issue</h3>

        <input
          type="text"
          placeholder="Issue title"
          value={this.state.title}
          onChange={this.handleChange}
        />

        <button onClick={this.handleAdd}>
          Add
        </button>
      </div>
    );
  }
}


class IssueList extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      issues: [
        {
          id: 1,
          title: "Login page not working"
```

```
        },
        {
          id: 2,
          title: "Database connection error"
        },
        {
          id: 3,
          title: "Page loading slowly"
        }
      ],
      filterText: ""
    };
  }

  addIssue = (title) => {
    this.setState((previousState) => ({
      issues: [
        ...previousState.issues,
        {
          id: previousState.issues.length + 1,
          title: title
        }
      ]
    }));
  };

  filterIssues = (text) => {
    this.setState({
      filterText: text
    });
  };

  render() {
    const filteredIssues = this.state.issues.filter(issue =>
      issue.title
        .toLowerCase()
        .includes(this.state.filterText.toLowerCase())
    );

    return (
      <div>
        <h2>Issue Tracker</h2>

        <IssueFilter
```

```
            onFilter={this.filterIssues}
          />

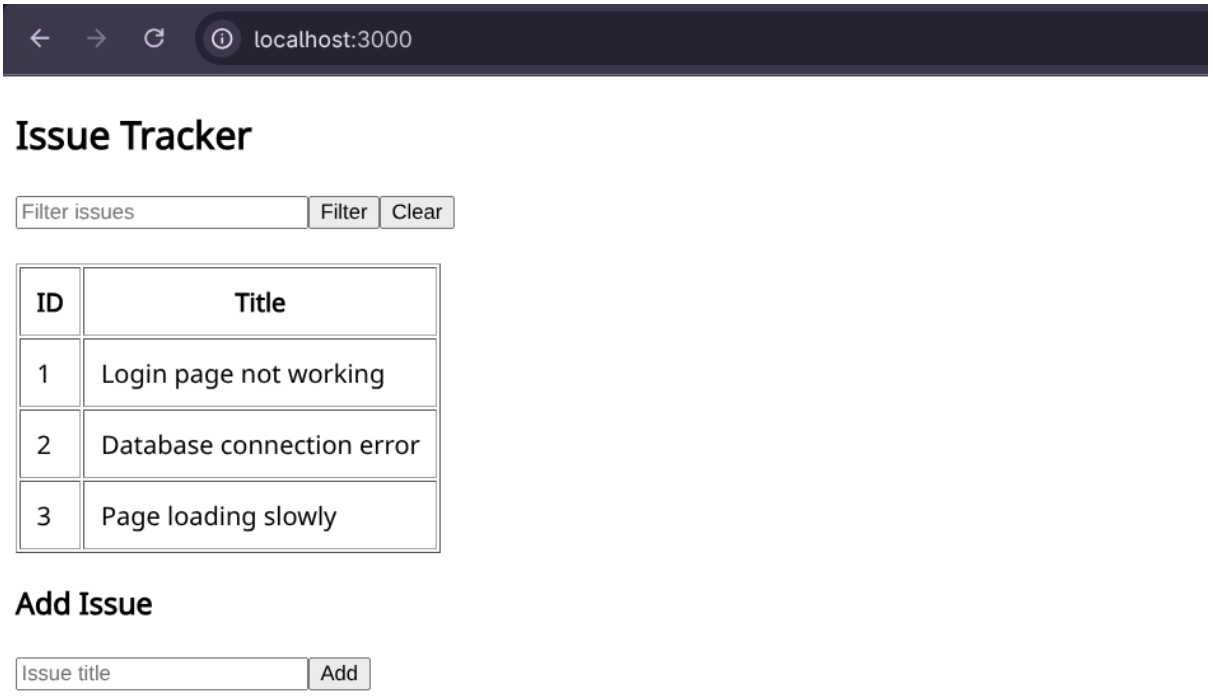
          <br />

          <IssueTable
            issues={filteredIssues}
          />

          <IssueAdd
            onAdd={this.addIssue}
          />
        </div>
      );
    }
}


ReactDOM.render(
  <IssueList />,
  document.getElementById("root")
);
```

Output:





Issue Tracker

Login Filter Clear

ID	Title
1	Login page not working

Add Issue

Issue title Add



Issue Tracker

Login Filter Clear

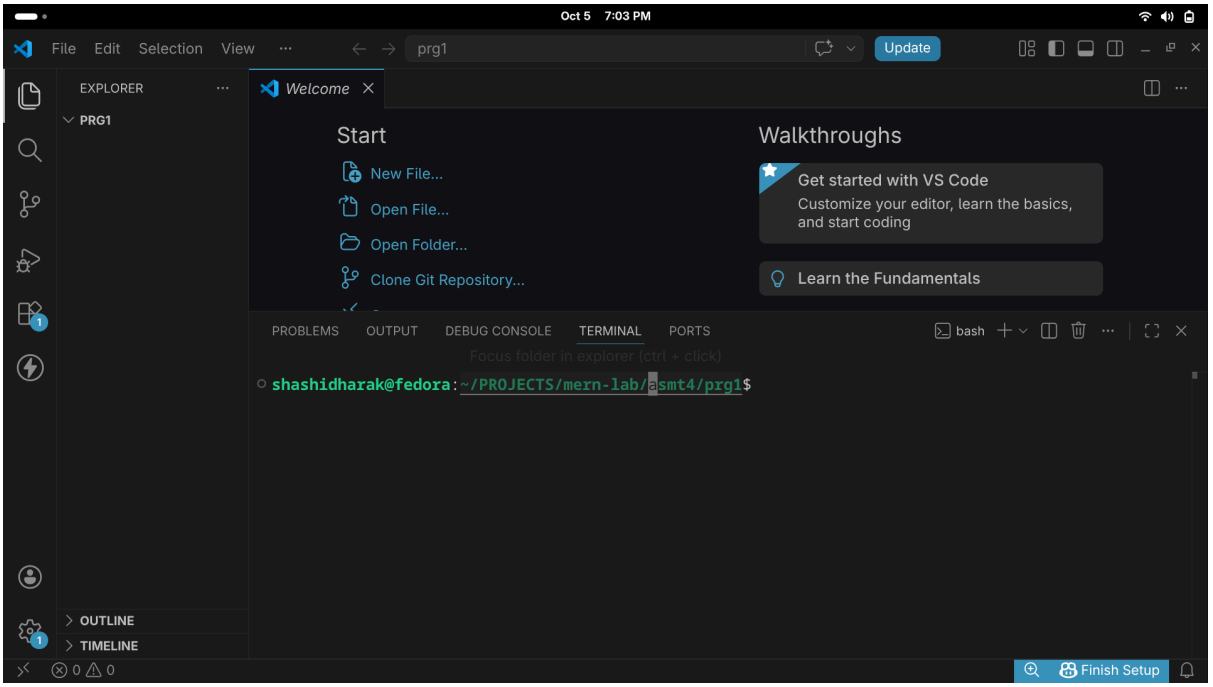
ID	Title
1	Login page not working
2	Database connection error
3	Page loading slowly
4	Server Error

Add Issue

Server Error Add

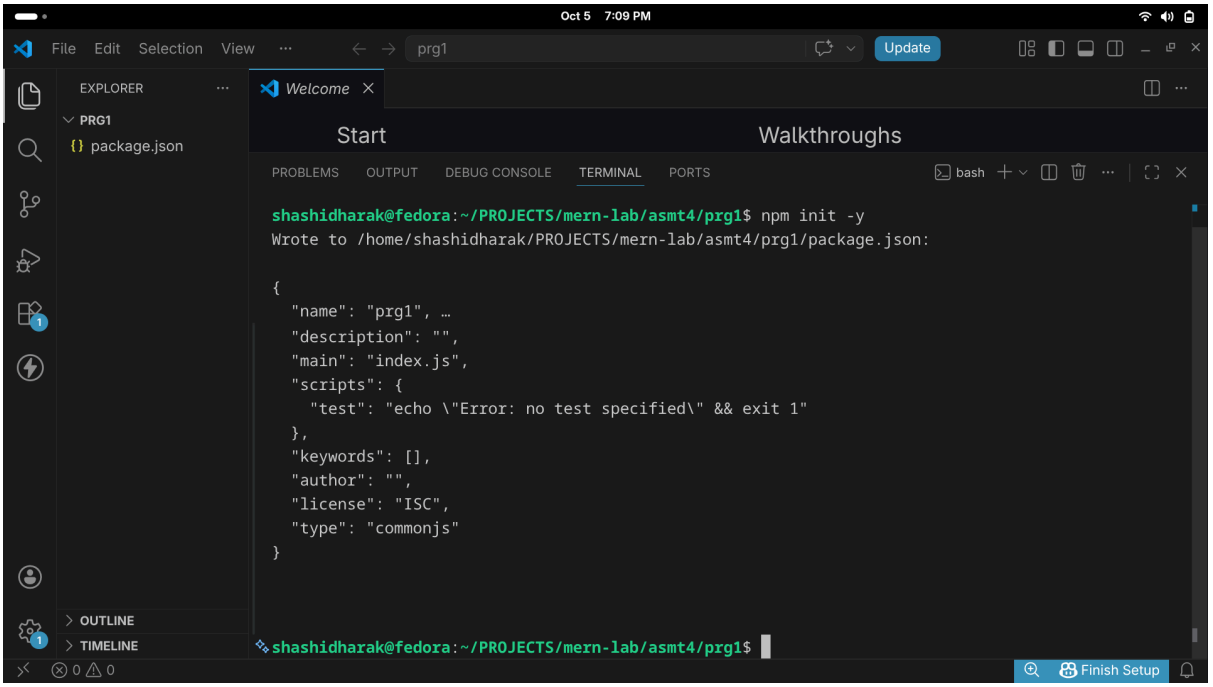
Example: How to run the program 1, (All 4 programs are similar,just changes has to be made in App.jsx , and rest all project structure we can copy and paste from previous program)

1. Create a folder prg1



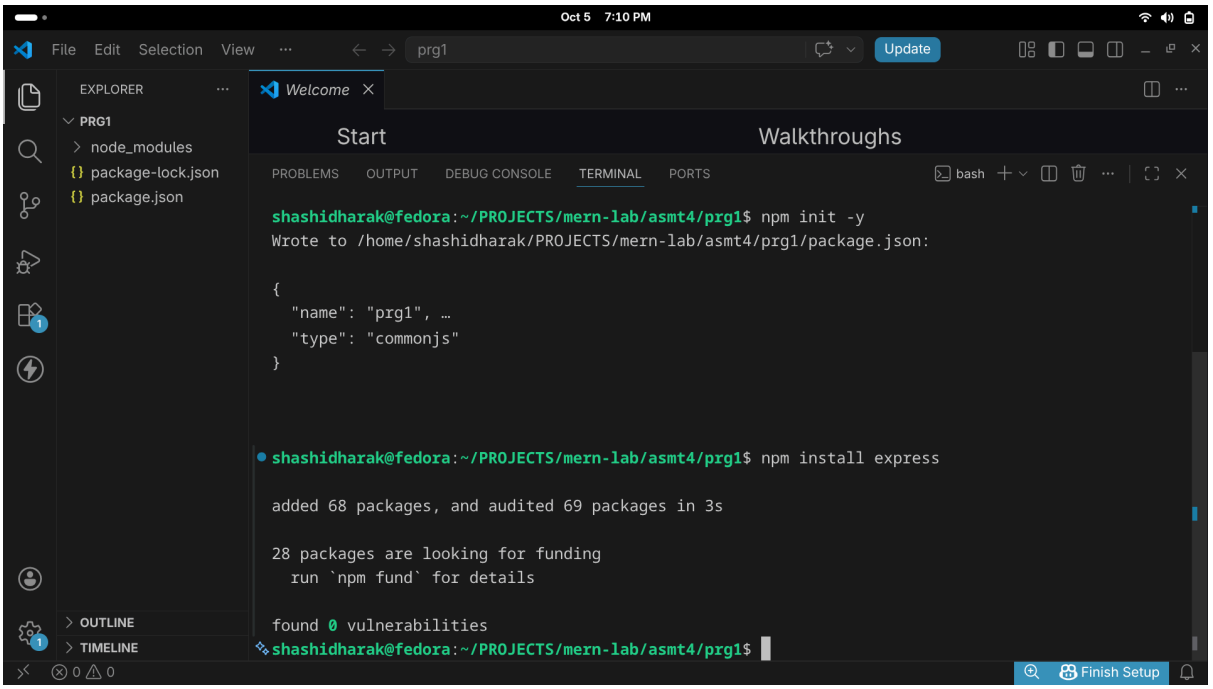
Open terminal and run

npm init -y



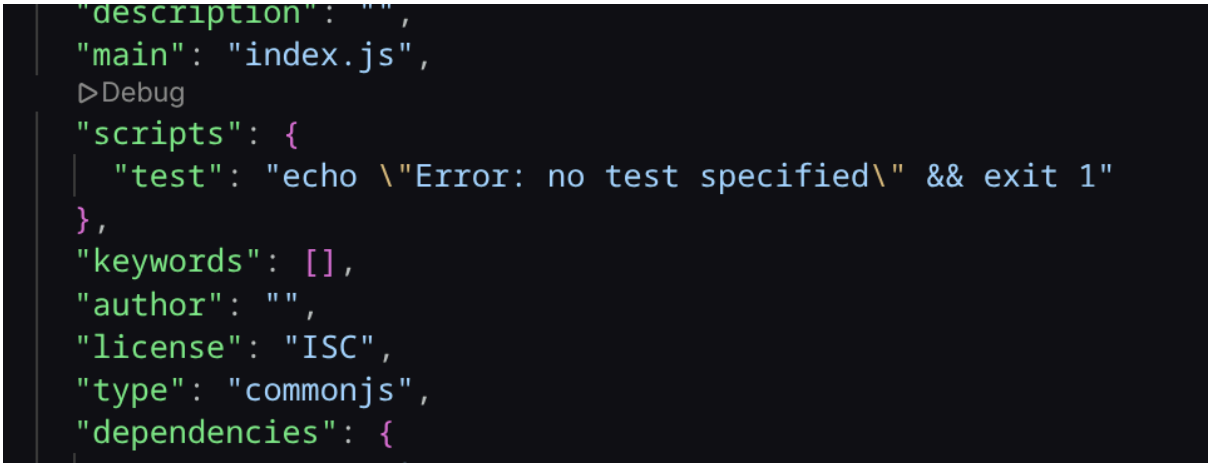
After that run one more command

npm install express



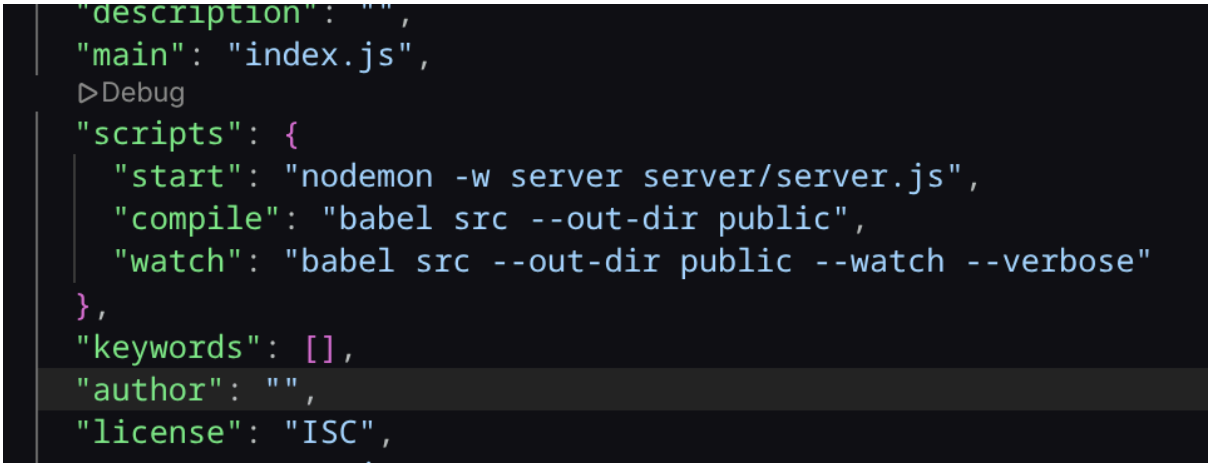
Make some modifications in package.json file , in the place of script

Original code:



Replace scripts with this code:

"scripts": {
"start": "nodemon -w server server/server.js",
"compile": "babel src --out-dir public",
"watch": "babel src --out-dir public --watch --verbose"
},



Make your project structure look like this:(Create folders and files manually which is missing - public/index.html src/App.jsx and server/server.js babel.config.json)



Paste the below codes inside those files,

package.json: Replace script code with this

```
"scripts": {
 "start": "nodemon -w server server/server.js",
 "compile": "babel src --out-dir public",
 "watch": "babel src --out-dir public --watch --verbose"
},
```

babel.config.json

```
{
 "presets": [
  [
   "@babel/preset-react",
   {
    "runtime": "classic"
   }
  ]
 ]
}
```

public/index.html

```
<!DOCTYPE html>
<html>

<head>
  <title>Hello World</title>
</head>

<body>

  <div id="root"></div>

  <script src="https://unpkg.com/react@17/umd/react.development.js"></script>
  <script
src="https://unpkg.com/react-dom@17/umd/react-dom.development.js"></script>

  <script src="App.js"></script>

</body>

</html>
```

server/server.js

```
const express = require('express');

const app = express();

app.use(express.static('public'));

app.listen(3000, function () {
  console.log('Server started at http://localhost:3000');
});
```

src/App.jsx

```
class HelloWorld extends React.Component {
  render() {
    const countries = [
      { id: 1, name: "India" },
      { id: 2, name: "Ireland" },
```

```
      { id: 3, name: "Canada" }
    ];

    const message = countries.map(c => (
      <CountryList key={c.id} country={c} />
    ));

    return message;
  }
}

class CountryList extends React.Component {
  render() {
    const row = this.props.country;

    return (
      <p>Hello World from {row.name}</p>
    );
  }
}

ReactDOM.render(
  <HelloWorld />,
  document.getElementById('root')
);
```

Open one terminal and run :

npm install --save-dev @babel/core @babel/cli @babel/preset-env @babel/preset-react nodemon

npm run compile

```
shashidharak@fedora:~/PROJECTS/mern-lab/asmt4/prg1$ npm install --save-dev @babel/
core @babel/cli @babel/preset-env @babel/preset-react nodemon

added 24 packages, and audited 234 packages in 4s

46 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
shashidharak@fedora:~/PROJECTS/mern-lab/asmt4/prg1$ npm run compile

> prg1@1.0.0 compile
> babel src --out-dir public

Successfully compiled 1 file with Babel (86ms).
shashidharak@fedora:~/PROJECTS/mern-lab/asmt4/prg1$
```

After execution of above command run this:

npm run watch

```
Successfully compiled 1 file with Babel (86ms).
shashidharak@fedora:~/PROJECTS/mern-lab/asmt4/prg1$ npm run watch

> prg1@1.0.0 watch
> babel src --out-dir public --watch --verbose

src/App.jsx -> public/App.js
Successfully compiled 1 file with Babel (78ms).
The watcher is ready.
```

Keep this terminal running, Open another terminal and run :

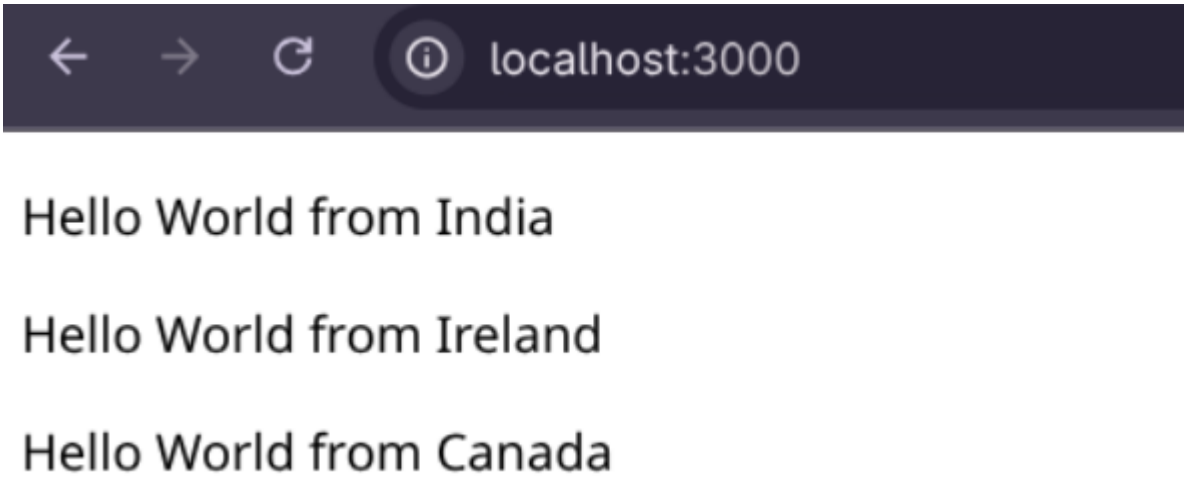
npm run start

```
shashidharak@fedora:~/PROJECTS/mern-lab/asmt4/prg1$ npm run start

> prg1@1.0.0 start
> nodemon -w server server/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): server/**/*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server/server.js`
Server started at http://localhost:3000
[nodemon] clean exit - waiting for changes before restart
```

Open http://localhost:3000 in the browser



Final package.json file looks like this

```
{
"name": "prg1",
"version": "1.0.0",
"description": "",
"main": "index.js",
"scripts": {
 "start": "nodemon -w server server/server.js",
 "compile": "babel src --out-dir public",
 "watch": "babel src --out-dir public --watch --verbose"
},
"keywords": [],
```

```
"author": "",
"license": "ISC",
"type": "commonjs",
"dependencies": {
  "express": "^5.2.1"
},
"devDependencies": {
  "@babel/cli": "^8.0.6",
  "@babel/core": "^8.0.6",
  "@babel/preset-env": "^8.0.6",
  "@babel/preset-react": "^8.0.1",
  "nodemon": "^3.1.14"
}
}
```